

Diseño Ágil

Semana 06

Ingeniería de Software 2

2026-1





Logro de la sesión



El estudiante será capaz de aplicar conceptos y principios de diseño ágil en la estructuración de software, utilizando técnicas de diseño de paquetes y tarjetas CRC para crear soluciones modulares y mantenibles que se adapten a los cambios del proyecto, es de manera efectiva en sistemas orientados a objetos.

Imagina que eres parte de un equipo de desarrollo de software que debe crear una aplicación innovadora, pero el mercado cambia rápidamente y los usuarios demandan nuevas características. Aquí es donde el diseño ágil se vuelve crucial, permitiendo al equipo iterar, recibir retroalimentación y ajustar su enfoque para alinearse con las necesidades del mercado.

En esta sesión, exploraremos cómo los conceptos de diseño ágil, los principios de diseño de paquetes y las tarjetas CRC son herramientas clave para adaptarse en un entorno dinámico. La capacidad de abrazar el cambio puede marcar la diferencia entre el éxito y el fracaso.

¿Estamos listos para ser arquitectos de soluciones que respondan a las necesidades de nuestros usuarios?



Contenidos

- Métodos Agiles
- Diseño Ágil
- Principios del diseño de paquetes

Métodos ágiles

Los métodos ágiles nacieron en la década de 1990 a partir de la necesidad de reducir los gastos generales aparentemente grandes asociados con los métodos pesados basados en planes utilizados en proyectos de desarrollo de software a gran escala.

Métodos ágiles

- Los métodos ágiles se consideran métodos ligeros
 - Ciclos de desarrollo breves e iterativos
 - Equipos autoorganizados
 - Diseños más simples
 - Refactorización de código
 - Desarrollo basado en pruebas
 - Participación frecuente del cliente
 - Énfasis en la creación de un entorno de trabajo transparente.



Manifiesto Ágil

- El [manifiesto Ágil](#) surge el 17 de febrero del 2001, cuando se reunieron diecisiete críticos del desarrollo de software, y acuñaron el término “metodología Ágil” para definir los métodos que estaban surgiendo como alternativa a las metodologías formales.
- El manifiesto Ágil está conformado por 12 principios asociados a 4 aspectos o pilares.

Aspectos o Pilares del Manifiesto

1 Satisfacción del cliente



Nuestra mayor prioridad es satisfacer al cliente mediante la entrega temprana y continua de software con valor

2 Bienvenidos los cambios



Aceptamos que los requisitos cambien, incluso en etapas tardías del desarrollo. Los procesos Ágiles aprovechan el cambio para proporcionar ventaja competitiva al cliente.

3 Entregas frecuentes

Sprint 1	Sprint 2	Sprint 3	Sprint 4
story	story	story	story
story	story	story	story
story	story	story	story

Entregamos software funcional frecuentemente, entre dos semanas y dos meses, con preferencia al periodo de tiempo más corto posible.

4 Trabajamos juntos



Los responsables de negocio y los desarrolladores trabajamos juntos diariamente durante todo el proyecto.

Aspectos o Pilares del Manifiesto

5 Confianza y apoyo



Los proyectos se desarrollan en torno a individuos motivados. Hay que darles el entorno y el apoyo que necesitan, y confiarles la ejecución del trabajo.

6 Conversaciones cara a cara



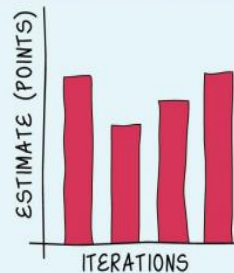
El método más eficiente y efectivo de comunicar información al equipo de desarrollo y entre sus miembros es la conversación cara a cara.

7 Software funcionando



El software funcionando es la medida principal de progreso

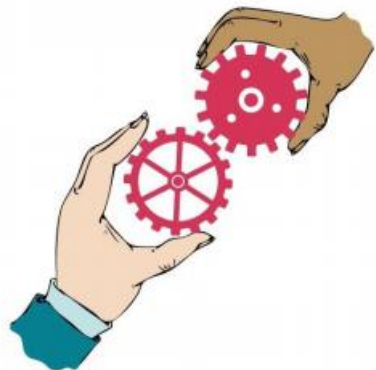
8 Desarrollo sostenible



Los procesos ágiles promueven el desarrollo sostenible. Los promotores, desarrolladores y usuarios debemos ser capaces de mantener un ritmo constante de forma indefinida.

Aspectos o Pilares del Manifiesto

9 Atención continua



La atención continua a la excelencia técnica y al buen diseño mejora la agilidad.

10 La simplicidad



El arte de maximizar la cantidad de trabajo no realizado, es esencial.

11 Equipos autoorganizados



Las mejores arquitecturas, requisitos y diseños emergen de equipos autoorganizados

12 Reflexionar y ajustar



A intervalos regulares el equipo reflexiona sobre cómo ser más efectivo para a continuación ajustar y perfeccionar su comportamiento en consecuencia.

Declaración de Interdependencia

- La [Declaración de Interdependencia](#) en la gestión de proyectos fue escrita a principios del 2005 por un grupo de 15 líderes de proyectos como un suplemento al “Manifiesto Ágil”. Enumera seis valores de gestión necesarios para reforzar una mentalidad de desarrollo ágil, particularmente en la gestión de proyectos **complejos e inciertos**.
- Los 6 Valores de la Declaración de Interdependencia
 1. Aumentamos el retorno de inversión, al enfocarnos en el flujo continuo de valor.
 2. Ofrecemos resultados fiables mediante la participación del cliente en las iteraciones frecuentes, donde también son responsables por el trabajo.
 3. Asumimos que habrá incertidumbre y las superamos a través de iteraciones, anticipación y adaptación.
 4. Damos rienda suelta a la creatividad y la innovación al reconocer que las personas son la fuente máxima de valor y creamos un entorno en el que puedan tener un impacto positivo.
 5. Aumentamos el rendimiento a través de la rendición de cuentas por parte del grupo en cuestión de resultados y eficacia del equipo, responsabilidades que todos comparten.
 6. Mejoramos la eficacia y la fiabilidad a través de estrategias situacionalmente específicas, procesos y prácticas.

¿Qué es Agilidad?

- “Agilidad es la capacidad de crear y responder al cambio con el fin de obtener ganancias en un entorno empresarial turbulento”.
- “La agilidad es la capacidad de equilibrar la flexibilidad y estabilidad”



¿Qué es Agilidad?

¿Cómo debemos ver a la Agilidad?

- En cualquier tipo de disciplina de gestión, ser ágil es una cualidad, por lo tanto esto debe ser una meta que se debe tratar de alcanzar.
- La gestión de proyectos Agile especialmente, implica la adaptabilidad durante la creación de un producto, servicio o cualquier otro resultado.

¿Por qué Metodologías Ágiles?

- El 80% de todos los proyectos emplearán Métodos Ágiles en los próximos años (Gartner).
- Casi tres cuartas partes (71%) de las organizaciones informan que utilizan enfoques ágiles a veces, a menudo o siempre



Gestión de Proyectos Tradicional



- Ventajas: Orden lógico.
- Desventaja: Asume predictibilidad.

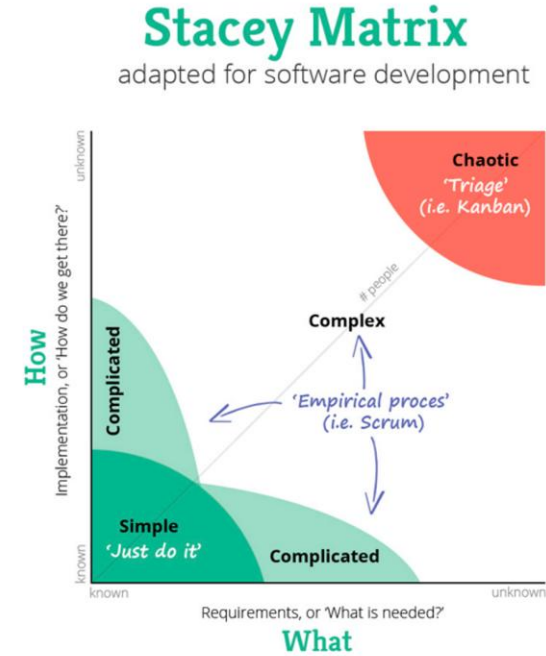


Agilidad y Complejidad



Professor of Management, Ralph D. Stacey

<https://Inds.net/blog/Inds/2018/07/08/agilidad-y-complejidad/>



Original Stacey Matrix by Ralph D. Stacey. Adapted by Christaan Verschuip (blog.agilistic.nl)

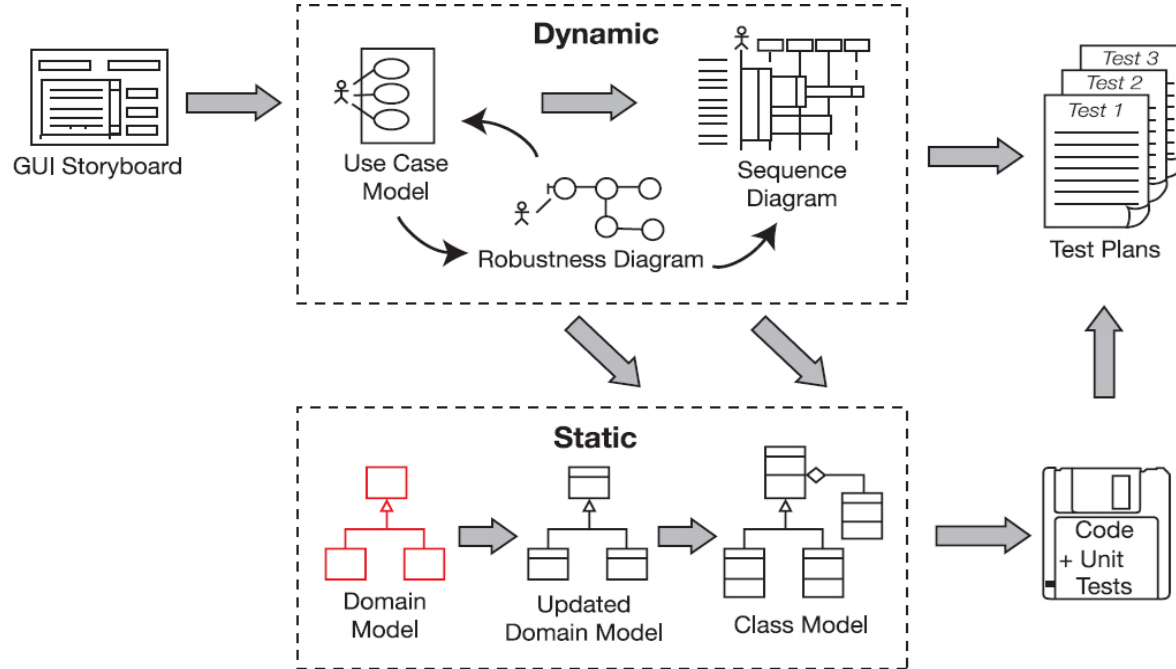
Términos clave relacionados con los procesos de Scrum

Backlog	Una lista de tareas que deben completarse como parte del proyecto. Esta lista es priorizada por los miembros del equipo al inicio de cada sprint. El backlog permite al equipo comunicar prioridades con un cliente y predecir con precisión la línea de tiempo de un proyecto.
Sprint	Un corto intervalo de tiempo (a menudo dos semanas) donde el equipo decide sobre un conjunto de tareas atrasadas para lograr como equipo. En la Figura 1 se muestra un tablero de sprint de ejemplo, que representa la acumulación y los elementos completados en un proyecto. Un ejemplo de equipo sprint se muestra en la Figura 2.
Retrospectiva Sprint	Como en todos los procesos ágiles, la reflexión es una parte importante de Scrum. Al final de cada sprint, el equipo de Scrum toma tiempo para revisar cómo fueron los procesos y hacer planes para mejorar los procesos en el futuro. Hacen preguntas como: “¿Qué hicimos bien y qué debería continuar?” o “¿Qué podríamos mejorar?”.
Ponte de pie	Una reunión diaria que está diseñada para durar 15 minutos o menos para actualizar al equipo sobre logros, problemas y estado. Se llama stand up porque está destinado a ser breve al tener a todos parados durante la reunión. Durante la reunión, los integrantes del equipo hacen preguntas como: “¿Qué hice ayer?” , “¿Estoy bloqueado por algo?” , o “¿Qué planeo hacer hoy?”.
Maestro Scrum	La persona que dirige el equipo de Scrum que se asegura de que todos los miembros del equipo obtengan los recursos que necesitan y se adhieren al plan del equipo.
Definición de Hecho	Este es un nivel de fidelidad acordado para la producción de productos en cada sprint. El equipo debe acordar cuál es la expectativa del trabajo de cada miembro del equipo.
Propietario del producto	Esta es la persona responsable del atraso. Trabajan para desarrollar una línea de tiempo precisa y mantener el proyecto encaminado. El Propietario del Producto no puede ser el Scrum Master.
Equipo Scrum	Todas las personas involucradas con el diseño del producto. Esto podría incluir desarrolladores, diseñadores de UX, QA y diseñadores instructivos, dado el proyecto. Diferentes sprints podrían tener diferentes miembros del equipo.

XP eXtreme Programming

- La metodología ágil más conocida y más utilizada es la programación extrema (XP)
- Es la única metodología ágil que se centra principalmente en el lado de la programación del desarrollo de software.
- Complemento de SCRUM ya que nos permite definir prácticas y técnicas:
 - Pruebas automáticas
 - Programación en pares
 - Refactoring

ICONIX un ejemplo de XP



Extreme Programming versus Scrum

Ventajas y Desventajas

- Extreme Programming como su nombre indica se podría considerar, pues ... extrema
- Scrum, por otra parte, es más fácil de digerir
- Scrum es menos técnico, menos detallada y menos estricta
 - Puede causar que los ingenieros se pierdan ó
 - Puede hacer libres a los ingenieros (menos es más!)
- Las prácticas de Extreme Programming son altamente dependientes
 - Que hace más difícil para la adopción gradual
- Las prácticas de Extreme Programming dependen de excelentes profesionales

Mejor Juntas XP y Scrum

- XP y Scrum no compiten, se complementan...
- Primero aplican prácticas XP y luego Scrum
 - Algunas de las prácticas de XP podrían utilizarse de manera aislada (incluso en el modelo de cascada!)
 - Después de ganar la confianza de TDD y refactoring la organización está más abierto a escuchar acerca de las alternativas de planificación (Scrum)
- Scrum primero y luego completar con XP
 - Las organizaciones que ha instalado correctamente scrum podrían apoyarse en las prácticas de ingeniería (XP)

Diseño Ágil

El diseño no es una fase, es una actividad continua.



Conceptos de diseño ágil

Diseño Ágil

El **diseño ágil** es un enfoque que busca construir sistemas de software de manera **incremental, adaptable y evolutiva**, evitando el diseño excesivo al inicio del proyecto.

A diferencia del enfoque tradicional (**Big Design Up Front - BDUF**), donde se define toda la arquitectura desde el inicio, el diseño ágil propone:

- Diseñar **solo lo necesario en el momento adecuado** (*Just-in-Time Design*)
- Permitir que el diseño **evolucione con los requisitos**
- Adaptarse continuamente al cambio

Síntomas de un diseño deficiente

- **Rigidez:** El sistema es difícil de cambiar porque cada cambio obliga a muchos otros cambios a otras partes del sistema.
- **Fragilidad:** los cambios hacen que el sistema se rompa en lugares que no tienen una relación conceptual con la parte que se cambió.
- **Inmovilidad:** es difícil separar el sistema en componentes que puedan reutilizarse en otros sistemas.
- **Viscosidad:** Hacer las cosas bien es más difícil que hacer las cosas mal.
- **Complejidad innecesaria:** el diseño contiene infraestructura que no agrega ningún beneficio directo.
- **Repetición innecesaria:** el diseño contiene estructuras repetitivas que podrían unificarse bajo una sola abstracción.
- **Opacidad:** es difícil de leer y comprender. No expresa bien su intención.

Smells y Principios

- **Code smells:** (Código que huele o apesta) es cualquier síntoma en el código fuente de un programa que posiblemente indica un problema más profundo.
- **Un diseño smells** es un síntoma, es algo que se puede medir objetivamente. A menudo, el smells es causado por la violación de uno o más de los principios SOLID.
- ¿Qué es el diseño de software? : [Jack W. Reeves](#) argumenta que el diseño de un sistema de software está documentado principalmente por su código fuente. Resulta que el artículo de Jack fue un presagio del **desarrollo ágil**.
- Un conjunto de diagramas UML puede representar partes de un diseño, pero no es el diseño. El diseño de un proyecto de software es un concepto abstracto.



¿Qué estimula el software a pudrirse?

- En entornos no ágiles, los diseños se degradan porque los requisitos cambian de formas que el diseño inicial no anticipó.
- A menudo, estos cambios deben realizarse rápidamente y pueden ser realizados por desarrolladores que no están familiarizados con la filosofía de diseño original.
- Entonces, aunque el cambio en el diseño funciona, de alguna manera viola el diseño original.
- Poco a poco, a medida que continúan los cambios, estas violaciones se acumulan y el diseño comienza a oler mal.
- De alguna manera debemos encontrar una manera de hacer que nuestros diseños sean resistentes a tales cambios y emplear prácticas que los protejan de la descomposición.

Los equipos ágiles no permiten que el software se pudra

- Un equipo ágil prospera con el cambio.
- El equipo invierte poco por adelantado; por lo tanto, no pertenece a un diseño inicial envejecido.
- Mantienen el diseño del sistema lo más limpio y simple posible, y lo respaldan con muchas pruebas unitarias y pruebas de aceptación.
- Esto mantiene el diseño flexible y fácil de cambiar.
- El equipo aprovecha esa flexibilidad para mejorar continuamente el diseño de modo que cada iteración finalice con un sistema cuyo diseño sea lo más apropiado posible para los requisitos de esa iteración.

Principios de Diseño en Ágil

Los principios guían decisiones de diseño simples y efectivas:

- **DRY (Don't Repeat Yourself)** : Evitar duplicación de lógica. Cada conocimiento debe tener una única representación.
- **KISS (Keep It Simple, Stupid)** : Preferir soluciones simples sobre complejas.
- **YAGNI (You Aren't Gonna Need It)** : No implementar funcionalidades hasta que sean realmente necesarias.
- **Bajo acoplamiento y alta cohesión** : Bajo acoplamiento, pocas dependencias entre clases. Alta cohesión, clases con responsabilidades claras
- **Ley de Demeter**: Un objeto solo debe comunicarse con sus colaboradores directos.
- **SOLID**: Conjunto de principios que promueven diseño flexible y mantenible: SRP, OCP, LSP, ISP, DIP

Diseño Emergente

El diseño emergente es un enfoque donde la arquitectura y la estructura del sistema no se definen completamente al inicio, sino que evolucionan gradualmente a medida que se desarrollan funcionalidades.

Se basa en la idea de que:

“El mejor diseño surge a partir de la experiencia, retroalimentación y refactorización continua.”

Objetivo

- Adaptarse a cambios de requisitos
- Evitar sobre-diseño
- Mantener el sistema simple y flexible
- Permitir aprendizaje progresivo del dominio



Diseño Emergente vs Diseño Anticipado (BDUF)

Diseño Anticipado (BDUF)	Diseño Emergente
Se define todo al inicio	Evoluciona continuamente
Alto riesgo de error	Se ajusta con feedback
Difícil de cambiar	Flexible
Basado en suposiciones	Basado en evidencia

Principio clave: “Diseñar lo suficiente para hoy, mejorar mañana.”

Esto implica:

- No anticipar necesidades futuras innecesarias (YAGNI)
- Construir soluciones simples
- Refinar el diseño conforme crece el sistema

Las 4 Reglas del Diseño Simple (Kent Beck)

Un diseño es bueno si cumple:

1. El sistema funciona correctamente

- Cumple todos los requisitos
- Todas las pruebas pasan

2. No tiene duplicación (DRY)

- Elimina código repetido
- Centraliza lógica

3. Es claro y expresivo

- Código fácil de entender
- Nombres significativos
- Intención evidente

4. Tiene el menor número de elementos

- Evita clases/métodos innecesarios

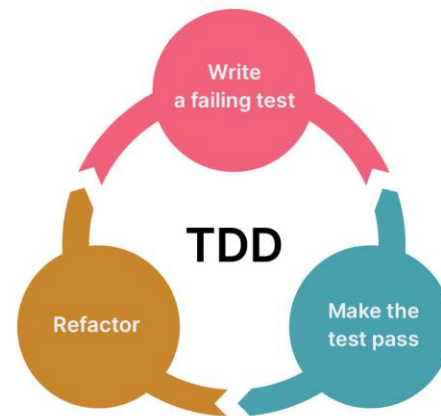


Relación con Refactorización

El diseño emergente **no funciona sin refactorización.**

Ciclo típico:

1. Implementar solución simple
2. Validar (tests)
3. Refactorizar
4. Mejorar diseño



TDD impulsa el diseño emergente porque:

- Obliga a escribir código mínimo
- Favorece bajo acoplamiento
- Promueve alta cohesión
- Hace visible cuándo refactorizar

TDD = motor del diseño emergente

El diseño mejora continuamente sin detener el desarrollo.

Arquitectura Ágil

La **arquitectura ágil** es un enfoque que propone que la arquitectura del sistema **no se define completamente al inicio**, sino que **evoluciona de manera incremental** a medida que el sistema crece.

Se alinea con los principios ágiles:

- Adaptabilidad
- Entrega continua
- Respuesta al cambio

Objetivos:

- Soportar cambios en los requisitos
- Reducir el riesgo técnico
- Permitir entregas tempranas
- Mantener calidad estructural del sistema

Arquitectura Tradicional vs Arquitectura Ágil

Tradicional	Ágil
Definida completamente al inicio	Evolutiva
Basada en predicciones	Basada en feedback
Difícil de cambiar	Flexible
Documentación extensa	Documentación ligera

Principios de la Arquitectura Ágil

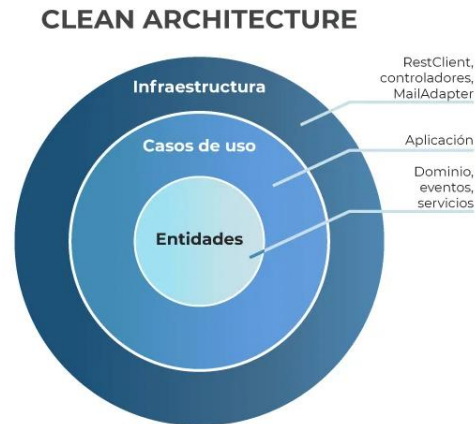
1. **Evolución continua** : La arquitectura se adapta conforme se desarrollan nuevas funcionalidades.
2. **Decisiones Just-in-Time** : Las decisiones se toman **cuando hay suficiente información**, no antes.
3. **Suficiente arquitectura inicial** : Se define una base mínima:
 - Capas
 - Estilo arquitectónico
 - Tecnologías clave*“Just enough architecture”*
4. **Enfoque en calidad**
Se consideran atributos como:
 - Escalabilidad
 - Mantenibilidad
 - Rendimiento



Arquitectura Evolutiva

La arquitectura se mejora iterativamente mediante:

- Refactorización
- Feedback del usuario
- Nuevos requerimientos



Permite que el sistema crezca sin perder estructura.

Introducción CRC Cards

Las tarjetas CRC son una técnica utilizada en el diseño de software para ayudar a identificar y definir las clases en un sistema orientado a objetos. Esta técnica fue introducida por Ward Cunningham y Kent Beck como parte de su trabajo en el desarrollo ágil y en la programación orientada a objetos.

Componentes de una tarjeta CRC

Cada tarjeta CRC contiene tres secciones principales:

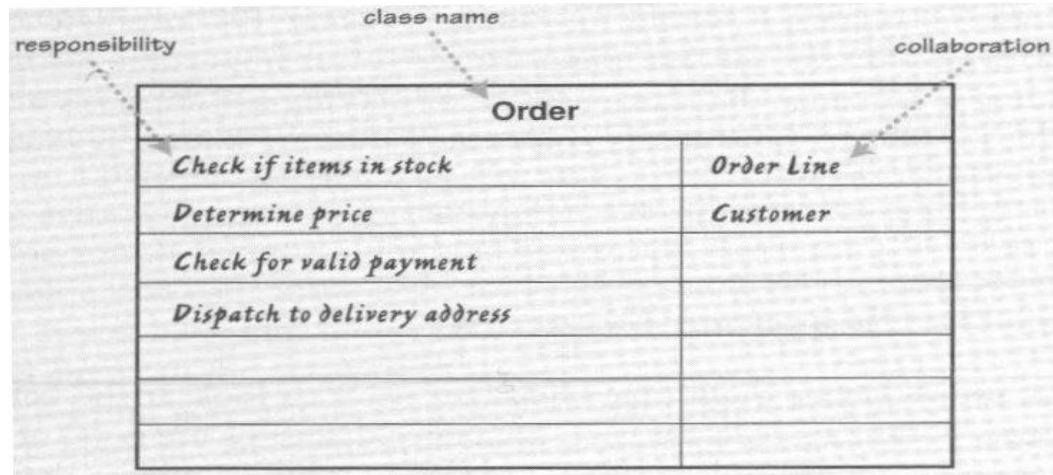
1. **Clase:** El nombre de la clase que se está definiendo. Este nombre debe ser descriptivo y reflejar la responsabilidad principal de la clase.
2. **Responsabilidades:** Una lista de las responsabilidades o funciones que la clase debe cumplir. Estas responsabilidades describen lo que la clase debe hacer y pueden incluir métodos y comportamientos específicos.
3. **Colaboradores:** Una lista de otras clases con las que esta clase interactúa o colabora para cumplir sus responsabilidades. Esto ayuda a identificar las relaciones y dependencias entre clases.

Uso de las tarjetas CRC

Las tarjetas CRC se utilizan en varias etapas del proceso de diseño de software:

- **Identificación de clases:** Durante las fases iniciales de diseño, las tarjetas CRC ayudan a identificar las clases necesarias en el sistema y a definir sus roles y responsabilidades.
- **Facilitación de la comunicación:** Al ser una herramienta visual y sencilla, las tarjetas CRC facilitan la discusión y el intercambio de ideas entre los miembros del equipo, promoviendo la colaboración y la comprensión compartida del diseño.
- **Refinamiento del diseño:** A medida que se avanza en el desarrollo, las tarjetas CRC pueden ser revisadas y refinadas para adaptarse a los cambios en los requisitos o en la comprensión del sistema.
- **Documentación:** Las tarjetas CRC sirven como una forma de documentación ligera que puede ser útil para nuevos miembros del equipo o para la revisión del diseño en el futuro.

CRC Card Template



The diagram shows a CRC Card Template for the class 'Order'. Annotations with dashed arrows point to specific parts of the card:

- responsibility**: Points to the first column of the table.
- class name**: Points to the header 'Order'.
- collaboration**: Points to the second column of the table.

Order	
<i>Check if items in stock</i>	<i>Order Line</i>
<i>Determine price</i>	<i>Customer</i>
<i>Check for valid payment</i>	
<i>Dispatch to delivery address</i>	

Proceso - CRC Card

- Básico: Simular la ejecución de escenarios de casos de uso / historias de usuarios
- Pasos:
 - Lluvia de ideas - class/object/component
 - Asignar class/object/component a personas (grupo hasta 6 personas)
 - Ejecuta los escenarios uno por uno
 - Añadir nuevas class/object/component según sea necesario
 - Añadir nuevas responsabilidades
 - delega a otras clases o personas

Ejemplo

Se está desarrollando un sistema de gestión para una tienda en línea. El objetivo es modelar las clases principales del sistema utilizando la técnica de tarjetas CRC (Class-Responsibility-Collaborator).

Clases a modelar:

- **Producto:** Esta clase representa los productos disponibles en la tienda. Debe almacenar información relevante como el nombre, la descripción, el precio y el stock. Sus responsabilidades incluyen permitir la actualización de la información del producto y proporcionar detalles para su visualización en la tienda. Colabora con la clase **Categoría** para clasificar los productos y con la clase **Carrito de Compras** para gestionar la adición de productos al carrito.
- **Carrito de Compras:** Esta clase gestiona los productos seleccionados por el usuario. Sus responsabilidades incluyen almacenar los productos, calcular el total de la compra y permitir la modificación de la cantidad de productos en el carrito. Colabora con la clase **Producto** para obtener información sobre los productos y con la clase **Usuario** para asociar el carrito con un usuario específico.
- **Usuario:** Esta clase representa a los usuarios del sistema. Debe almacenar información del usuario, como el nombre, la dirección y el historial de compras. Sus responsabilidades incluyen gestionar la autenticación y el registro del usuario, así como permitir la visualización y modificación de la información del perfil. Colabora con la clase **Carrito de Compras** para gestionar el carrito del usuario y con la clase **Pedido** para procesar las compras realizadas.
- **Pedido:** Esta clase almacena información sobre los pedidos realizados por los usuarios. Sus responsabilidades incluyen almacenar los productos comprados, el total de la compra y el estado del pedido. También gestiona el proceso de pago y proporciona detalles del pedido al usuario. Colabora con la clase **Usuario** para asociar el pedido con un usuario específico y con la clase **Producto** para obtener información sobre los productos comprados.

Solución

Tarjeta CRC 1: Producto

•**Clase:** Producto

•**Responsabilidades:**

- Almacenar información sobre el producto (nombre, descripción, precio, stock).
- Permitir la actualización de la información del producto.
- Proporcionar detalles del producto para su visualización en la tienda.

•**Colaboradores:**

- Categoría (para clasificar el producto).
- Carrito de Compras (para gestionar la adición de productos al carrito).

Solución

Tarjeta CRC 2: Carrito de Compras

•**Clase:** Carrito de Compras

•**Responsabilidades:**

- Almacenar los productos seleccionados por el usuario.
- Calcular el total de la compra.
- Permitir la modificación de la cantidad de productos en el carrito.

•**Colaboradores:**

- Producto (para obtener información sobre los productos en el carrito).
- Usuario (para asociar el carrito con un usuario específico).

Solución

Tarjeta CRC 3: Usuario

•**Clase:** Usuario

•**Responsabilidades:**

- Almacenar información del usuario (nombre, dirección, historial de compras).
- Gestionar la autenticación y el registro del usuario.
- Permitir la visualización y modificación de la información del perfil.

•**Colaboradores:**

- Carrito de Compras (para gestionar el carrito del usuario).
- Pedido (para procesar las compras realizadas por el usuario).

Solución

Tarjeta CRC 4: Pedido

•**Clase:** Pedido

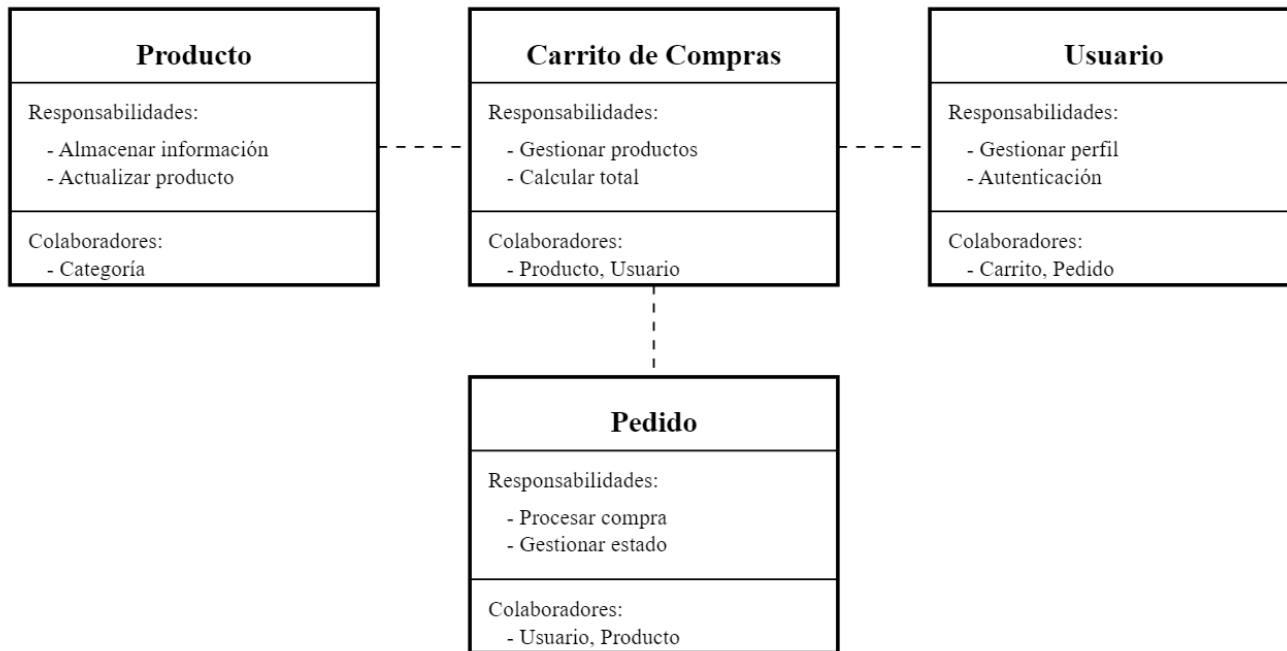
•**Responsabilidades:**

- Almacenar información sobre el pedido (productos, total, estado).
- Gestionar el proceso de pago y confirmación del pedido.
- Proporcionar detalles del pedido al usuario.

•**Colaboradores:**

- Usuario (para asociar el pedido con un usuario).
- Producto (para obtener información sobre los productos comprados).

Ejemplo



Principios del diseño de paquetes

Las clases, si bien son una unidad muy conveniente para organizar aplicaciones pequeñas, son demasiado finas para ser utilizadas como la única unidad organizativa para aplicaciones grandes.

Conceptos Clave

- **Paquete:** Un paquete es un contenedor que agrupa clases y otros paquetes relacionados. Facilita la organización del código y la gestión de dependencias.
- **Cohesión:** Se refiere a la medida en que los elementos dentro de un paquete están relacionados entre sí. Un paquete altamente cohesivo contiene clases que trabajan juntas para cumplir una única responsabilidad.
- **Acoplamiento:** Se refiere a la dependencia entre paquetes. Un bajo acoplamiento significa que los paquetes son independientes entre sí, lo que facilita su modificación y reutilización.
- **Encapsulamiento:** Este principio implica ocultar los detalles internos de un paquete y exponer solo lo necesario a otros paquetes. Esto protege la integridad del paquete y reduce el riesgo de errores.
- **Interfaz:** Cada paquete debe tener una interfaz bien definida que especifique cómo interactuar con él. Esto mejora la usabilidad y la integración con otros componentes.

Diseño de paquetes

Conforme el tamaño de las aplicaciones crece, se hace necesario algún tipo de organización a alto nivel.

Las clases son unidades demasiado pequeñas, por lo que se agrupan en **paquetes**.

¿Qué criterios utilizaremos para organizar las clases? Los mismos que para organizar los miembros de las clases: la **cohesión** y el **acoplamiento**

Principios del Diseño de Paquetes

- **Organización Lógica:** Agrupar clases en paquetes de manera que refleje la estructura lógica del sistema. Por ejemplo, se pueden crear paquetes para diferentes módulos funcionales, como "usuarios", "productos" y "pedidos".
- **Reutilización:** Diseñar paquetes para que sean reutilizables en diferentes contextos. Esto implica crear clases y métodos que puedan ser utilizados en múltiples aplicaciones o módulos.
- **Simplicidad:** Mantener los paquetes simples y enfocados en una única responsabilidad. Esto facilita la comprensión y el mantenimiento del código.
- **Minimizar Dependencias:** Reducir las dependencias entre paquetes para evitar que los cambios en un paquete afecten a otros. Esto se puede lograr mediante el uso de interfaces y la inyección de dependencias.
- **Documentación:** Proporcionar documentación clara sobre el propósito y el uso de cada paquete. Esto es esencial para que otros desarrolladores comprendan cómo utilizar y extender el paquete.

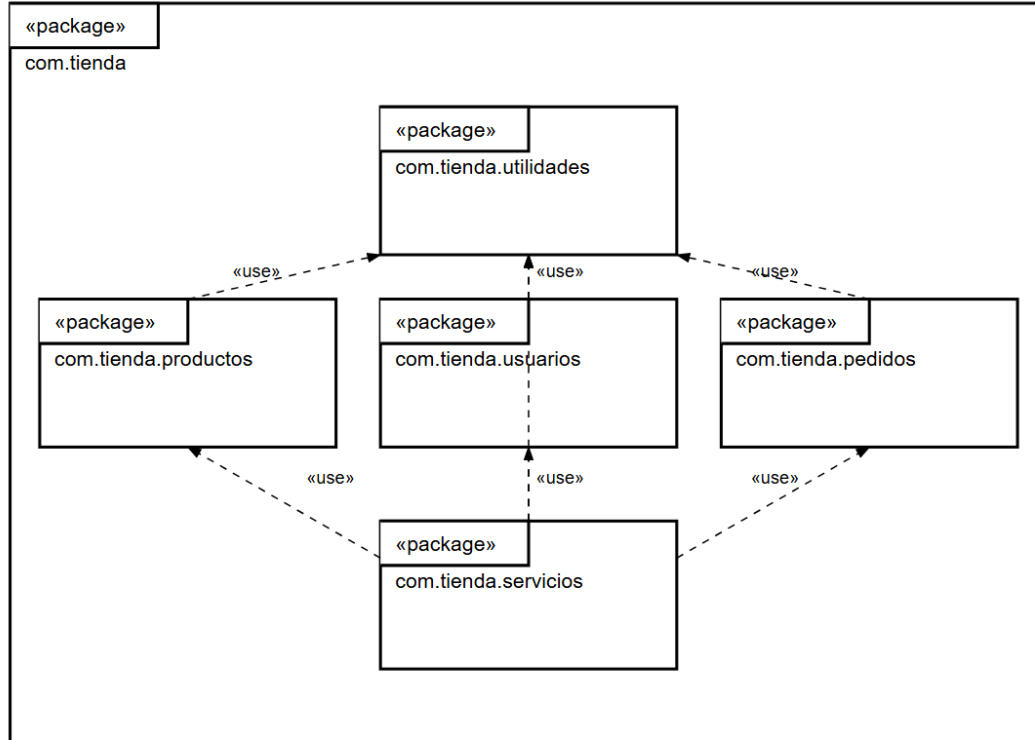
Ejemplo de Diseño de Paquetes

Imaginemos un sistema de gestión de una tienda en línea. Un posible diseño de paquetes podría ser el siguiente:

- **com.tienda.productos:** Contiene clases relacionadas con la gestión de productos, como Producto, Categoria, y Inventario.
- **com.tienda.usuarios:** Incluye clases para la gestión de usuarios, como Usuario, Perfil, y Autenticacion.
- **com.tienda.pedidos:** Agrupa clases relacionadas con la gestión de pedidos, como Pedido, Carrito, y Pago.
- **com.tienda.servicios:** Contiene clases de servicio que implementan la lógica de negocio, como ProductoService, UsuarioService, y PedidoService.
- **com.tienda.utilidades:** Incluye clases utilitarias que proporcionan funciones comunes, como Validador, Logger, y Configuracion.



Ejemplo de Diseño de Paquetes



Granularidad: La cohesión de los paquetes

¿Cuándo se ponen dos clases en el mismo paquete?

- Cuando para usar una, siempre es necesario usar la otra.
- Cuando dos clases están en un mismo paquete, es porque se usan juntas. Por tanto, todas las clases de un paquete se usan conjuntamente: si se usa una, se usan todas.

¿cuándo se ponen en paquetes diferentes?

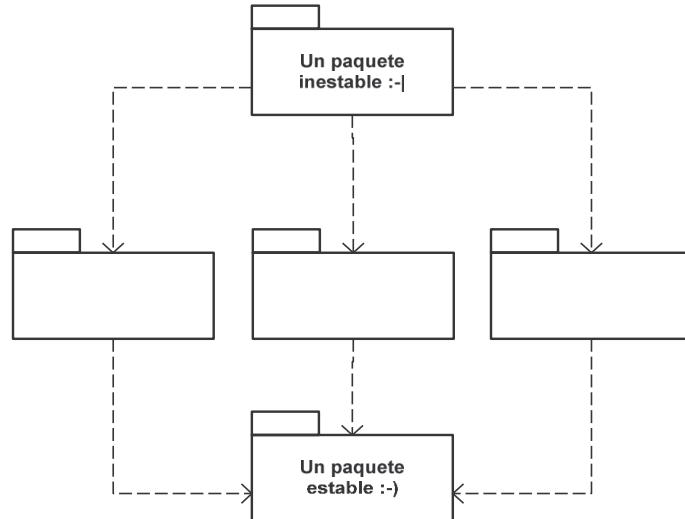
- Aquellas clases que no siempre se usen conjuntamente con las demás clases del paquete son candidatas para abandonar el paquete (quizá para ir a un subpaquete más específico).

Las clases de un paquete se verán afectadas por los mismos tipos de cambios y las modificaciones necesarias para realizar un cambio concreto deberán estar localizadas en un único paquete.

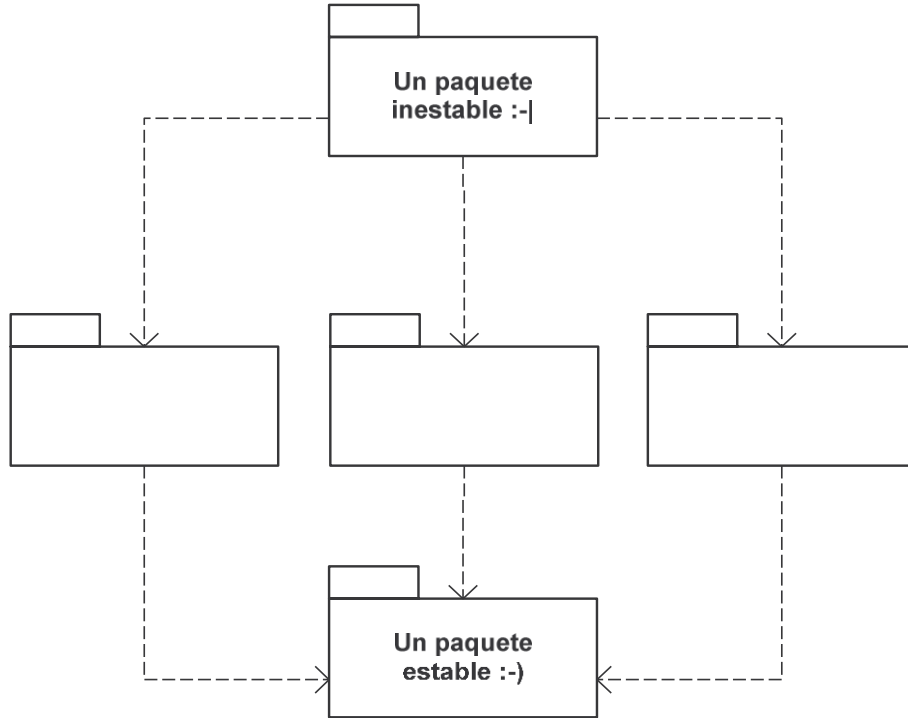
Estabilidad: El acoplamiento entre paquetes

Para facilitar el desarrollo de un sistema complejo, resulta aconsejable que las distintas partes del sistema sean lo más independientes posible:

Cuanto menos dependa un paquete de otros paquetes, mejor.



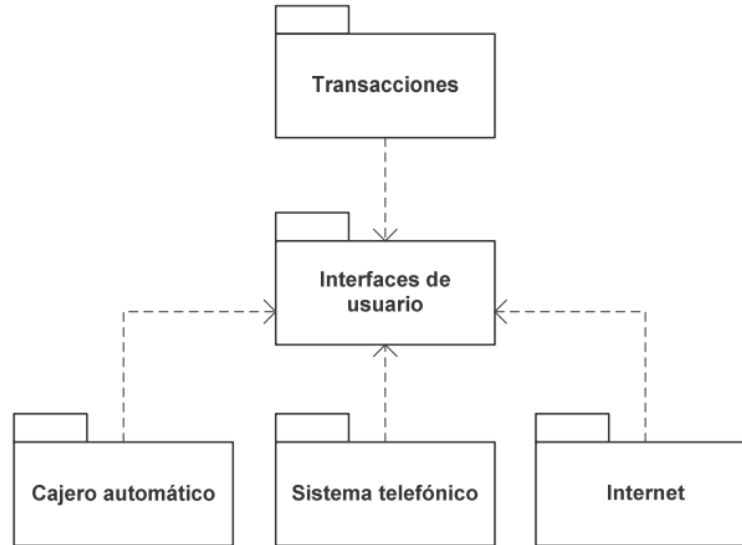
Estabilidad: El acoplamiento entre paquetes



- Un paquete es más estable cuando depende de **menos** paquetes.
- Cuando queramos que un paquete sea flexible (esto es, fácil de cambiar), mejor si hay **pocos paquetes** que dependan de él.
- Un paquete debe ser más estable cuanto **más abstracto** sea (un paquete estable y concreto se vuelve rígido).

Diseño de paquetes

Si dibujamos un diagrama con las dependencias existentes entre los paquetes, el diagrama no debe tener ciclos:

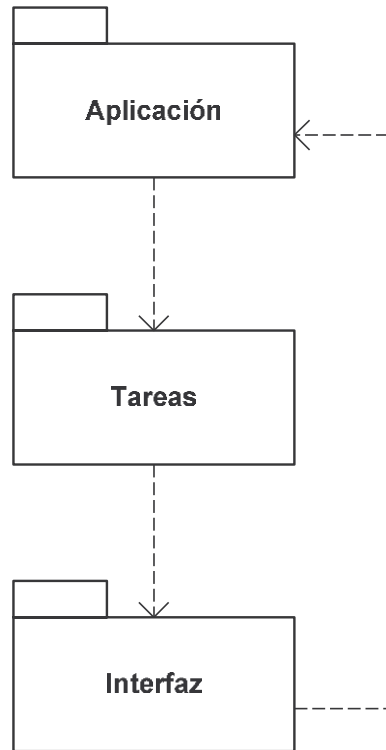


¿Qué efectos ocasiona un ciclo?

Cuando trabajamos en la construcción de la interfaz, hemos de disponer de ciertos servicios proporcionados por la aplicación.

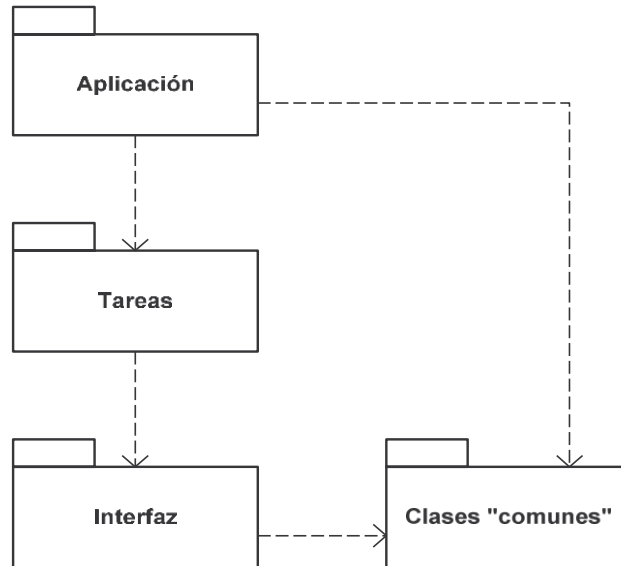
Esto hace que la interfaz dependa de todos los demás paquetes de la aplicación.

Para probar el funcionamiento de la interfaz necesitamos disponer de una implementación de todos los demás paquetes y las pruebas dependerán del estado actual de esos paquetes (por lo que difícilmente se pueden considerar *pruebas de unidad*).



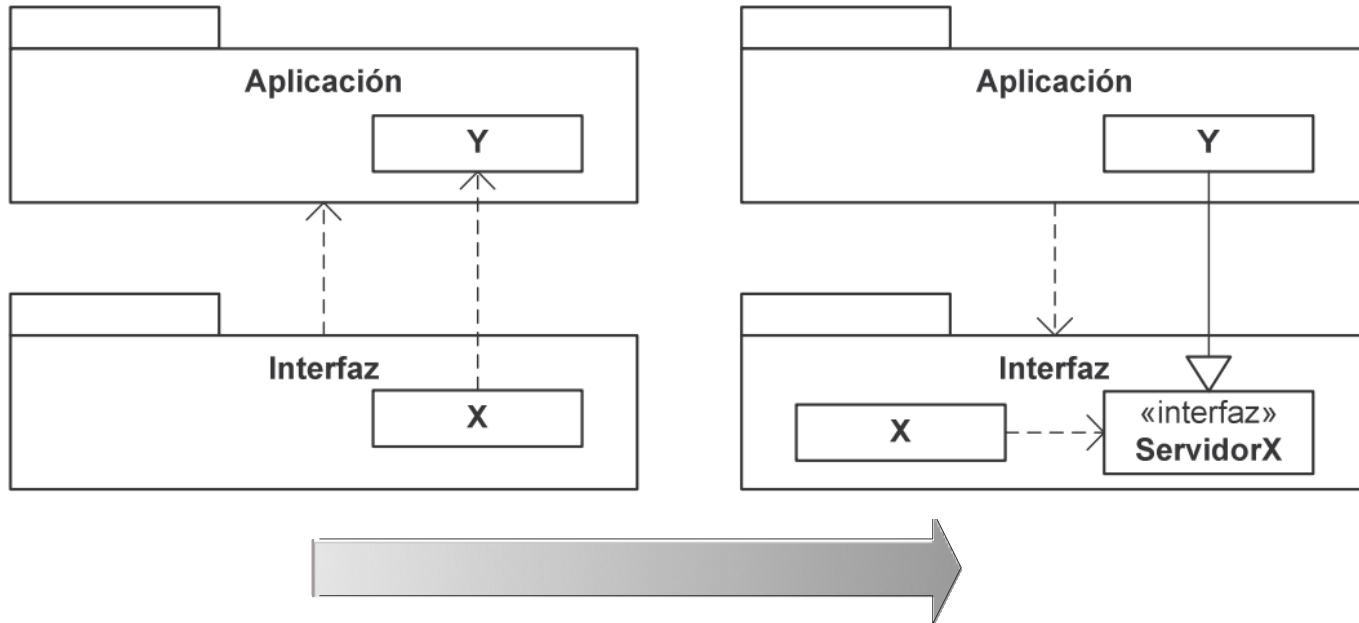
¿Cómo se rompe un ciclo entre dos paquetes?

1. Creando un nuevo paquete del que ambos dependan:



¿Cómo se rompe un ciclo entre dos paquetes?

2. Aplicando el principio de inversión de dependencias:



Casos de Estudio



Caso 1 :

Crear las tarjetas CRC para una biblioteca universitaria que necesita implementar un sistema para gestionar el préstamo de libros. El sistema debe permitir registrar cuando un usuario solicita el préstamo de un libro, considerando las siguientes reglas de negocio:

1.Reglas para Usuarios:

1. Solo pueden solicitar préstamos los usuarios registrados y activos
2. Un usuario puede tener máximo 9 libros prestados simultáneamente
3. El usuario no debe tener multas pendientes por retrasos

2.Reglas para Libros:

1. Solo se pueden prestar libros que estén disponibles (no prestados)
2. Algunos libros son solo para consulta en sala y no se pueden prestar
3. Cada libro tiene un período de préstamo según su categoría (7, 14 o 21 días)

3.Reglas para Préstamos:

1. Se debe registrar la fecha de préstamo y la fecha de devolución esperada
2. El sistema debe verificar si el usuario y el libro cumplen con las condiciones para realizar el préstamo
3. Se debe generar un comprobante del préstamo

Registrar el préstamo de un Libro a un usuario

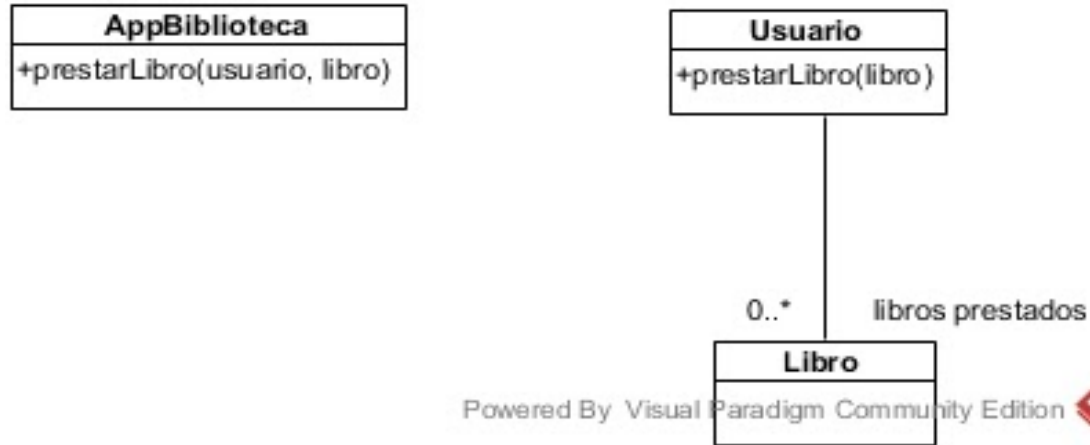
- Conjunto de tarjetas CRC iniciales: Aplicación Biblioteca, Usuario, Libro

Aplicación Biblioteca	
Registrar el préstamo de un libro a un usuario	Usuario Libro

Libro	

Usuario	
Prestarse un Libro Conocer los Libros prestados	

Prestar un Libro a usuario registrado



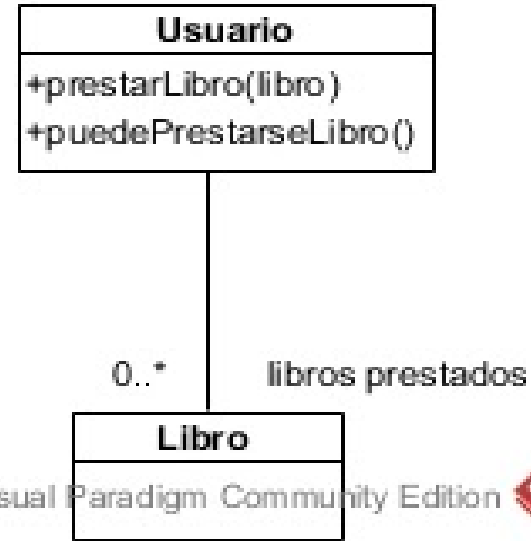
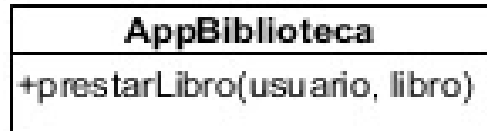
Prestar no más de 10 libros

Aplicación Biblioteca	
Registrar el préstamo de un libro a un usuario	Usuario Libro

Libro	

Usuario	
Prestarse un Libro Conocer los Libros prestados Conocer si puede prestarse	

Prestar no más de 10 libros



Powered By Visual Paradigm Community Edition 



Libro con fecha de devolución vencida

Aplicación Biblioteca	
Registrar el préstamo de un libro a un usuario Conocer la fecha actual	Usuario Libro Fecha

Libro	
Conocer si la fecha de devolución está Vencida Conocer la fecha de devolución	Fecha

Usuario	
Prestarse un Libro Conocer los Libros prestados Conocer si puede prestarse un libro	

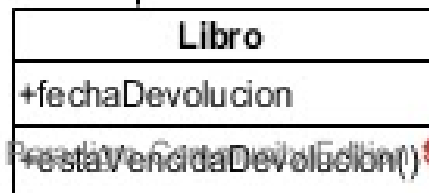
Fecha	
Comparar fechas	

Libro con fecha de devolución vencida



0..*

libros prestados

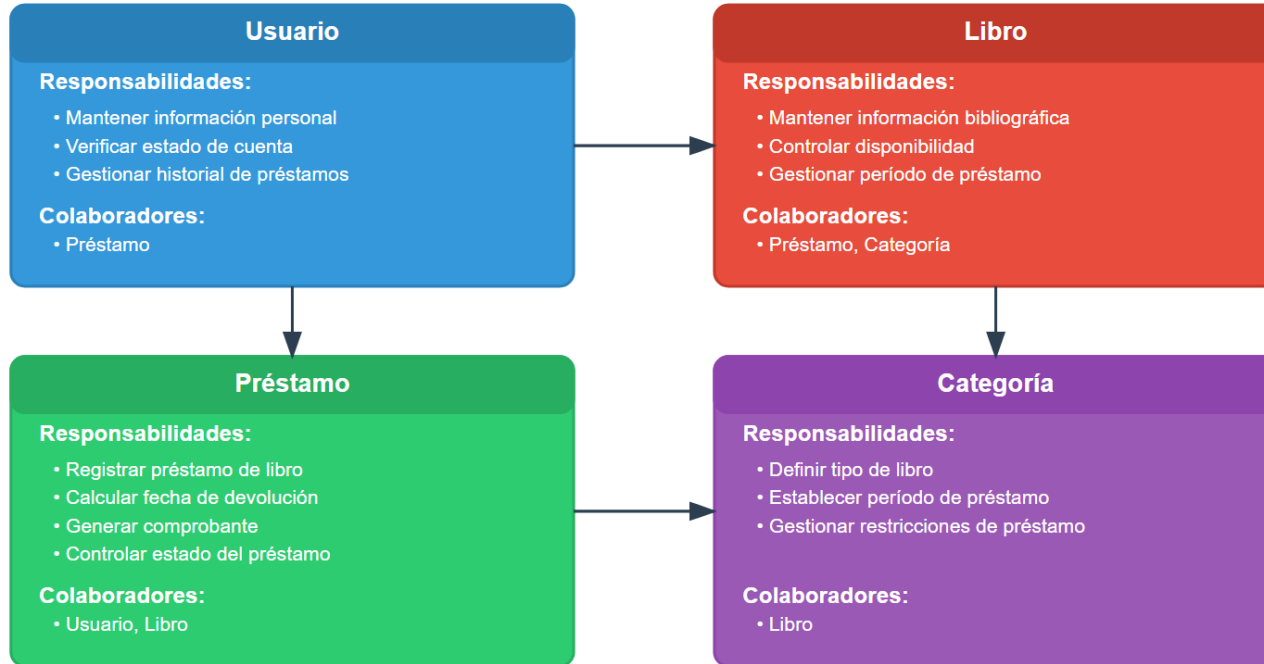


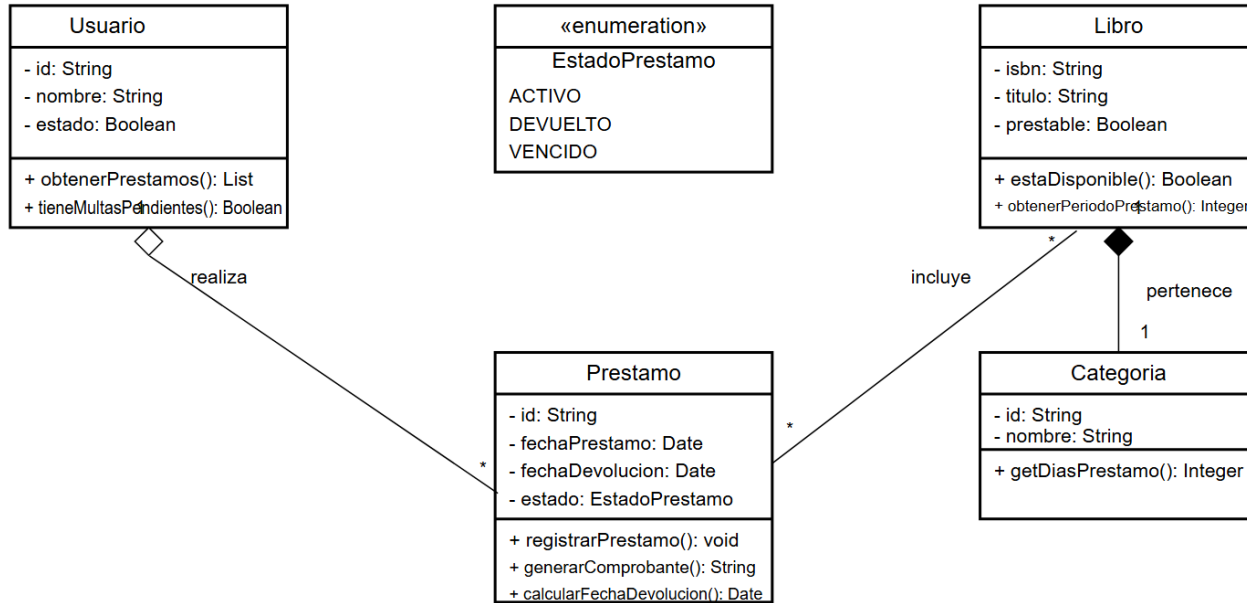
Powered By Visual Paradigm for UML



Solución :

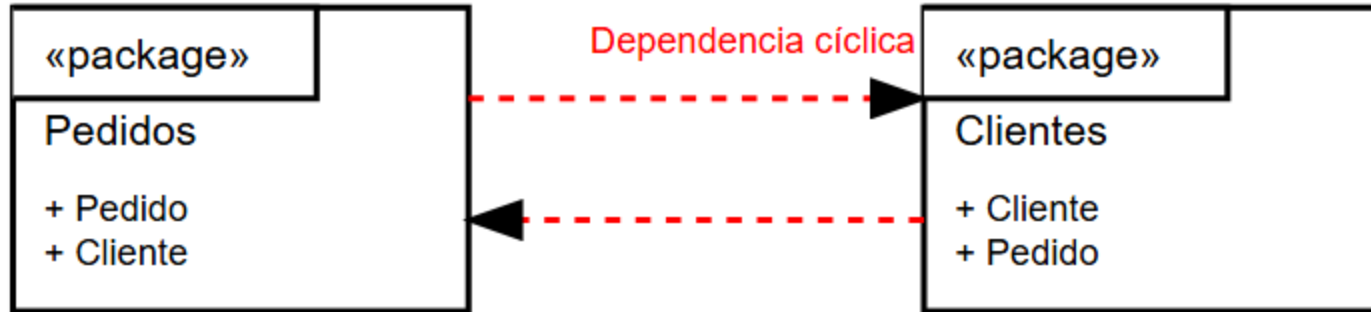
Tarjetas CRC - Sistema de Biblioteca





Caso 2 :

El siguiente diagrama de paquetes rompa el ciclo entre paquetes



Solucion :

Para romper un ciclo entre paquetes, se pueden aplicar las siguientes estrategias:

Crear una Abstracción Común:

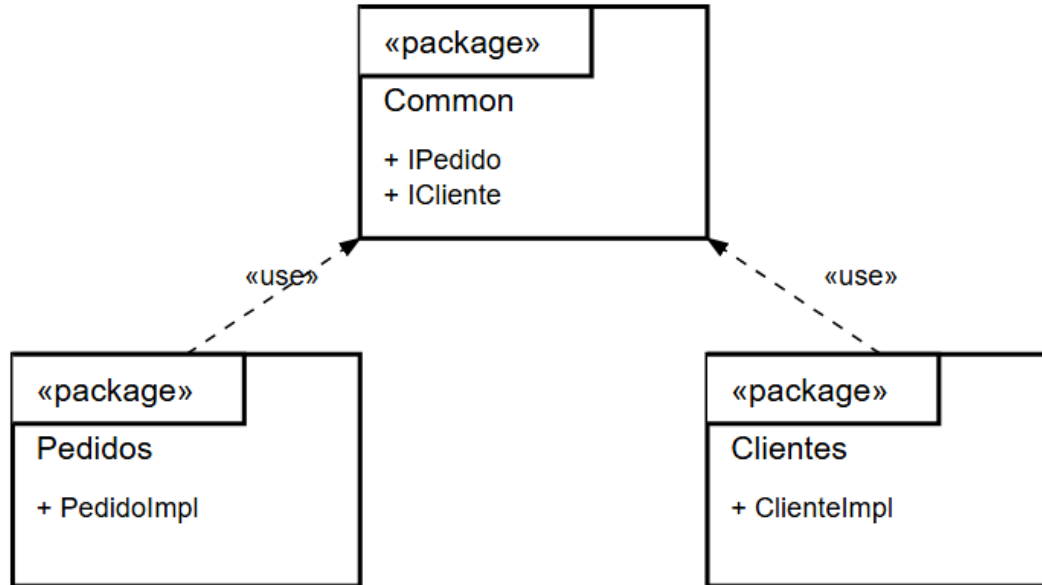
- Identificar las dependencias que causan el ciclo
- Extraer las interfaces o abstracciones comunes a un nuevo paquete
- Hacer que ambos paquetes dependan de esta abstracción común

Principio de Inversión de Dependencias (DIP):

- Los módulos de alto nivel no deben depender de módulos de bajo nivel
- Ambos deben depender de abstracciones



Solución :



¿Qué se logró?



- ✓ Comprensión sólida de los principios del diseño ágil y su relevancia en la adaptación a cambios en los requisitos del cliente durante el desarrollo de software.
- ✓ Aplicar efectivamente los principios de cohesión y acoplamiento en la organización de clases en módulos, lo que mejoró la mantenibilidad y escalabilidad de sus sistemas.
- ✓ Utilizar tarjetas CRC para identificar y definir clases, responsabilidades y colaboradores, lo que facilitó la comunicación y el diseño colaborativo en proyectos orientados a objetos.

¿Qué debemos hacer para alcanzar el objetivo del tema?



- ✓ Repasar la teoría, los ejemplos y casos resueltos.



Referencias Bibliográficas

- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1995). Design Patterns: Elements of Reusable Object-Oriented Software Addison-Wesley.
- <https://github.com/RameshMF/gof-java-design-patterns>
- <https://refactoring.guru/design-patterns/abstract-factory/java/example>
- <https://refactoring.guru/es/design-patterns/abstract-factory/java/example>
- <http://www.cs.sjsu.edu/faculty/pearce/modules/patterns/creational/factory.htm>
- <https://www.oscarblancarteblog.com/2014/07/18/patron-de-diseno-factory/>

GRACIAS